

Maximum Matchings & the Inverse FFT

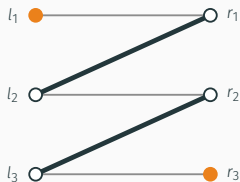
Lecture 8 | Algorithm Design & Analysis

IIIT Hyderabad

Recall: augmenting paths

Let M be a matching in G . A vertex is **matched** if some edge of M meets it, and **free** otherwise. An **alternating path** is a path whose edges lie alternately outside and inside M . An **augmenting path** is an alternating path whose two endpoints are both free.

Here $M = \{l_2r_1, l_3r_2\}$, drawn thick; l_1 and r_3 are free. The path $l_1r_1l_2r_2l_3r_3$ is augmenting.



An augmenting path has odd length $2k + 1$, with $k+1$ edges outside M and k inside it.

Lemma 1. If P is M -augmenting, then $M \oplus P$ is a matching and $|M \oplus P| = |M| + 1$.

Proof. The count follows from the previous paragraph. For the matching property, check each vertex: one off P is unchanged; an interior vertex of P exchanges its single M -edge for one other; an endpoint was free and gains one. ■

Comparing two matchings

Suppose M is not maximum, so a larger matching M^* exists. We cannot compute M^* , but we can still reason about the symmetric difference $M \oplus M^*$, the set of edges lying in exactly one of the two.

Every vertex meets at most one edge of M and at most one edge of M^* , so it has degree at most 2 in $M \oplus M^*$. A graph of maximum degree 2 is a disjoint union of paths and cycles. Moreover two edges meeting at a vertex cannot both lie in M , nor both in M^* , so the edges alternate along each component.



even cycle, no free end
 $\#M = \#M^*$



path, one end free
 $\#M = \#M^*$



path, both ends free
 $\#M^* = \#M + 1$: augmenting

— M - - M^* ● free in M

Berge's Lemma

M is a maximum matching if and only if M has no augmenting path.

($\Rightarrow$) If P is augmenting, Lemma 1 produces a larger matching, so M was not maximum.

($\Leftarrow$) Suppose M is not maximum, and take M^* with $|M^*| > |M|$.

1. Every cycle of $M \oplus M^*$ alternates, hence is even and contains equally many M and M^* edges. A path can differ by at most one.
2. M^* contributes more edges overall, so some component contains more M^* edges than M edges. By 1 that component is a path beginning and ending with an M^* edge.
3. Its endpoints are M -free. If an endpoint v met an edge $e \in M$, then either $e \in M \oplus M^*$, making v interior rather than an endpoint, or $e \in M \cap M^*$, giving v two M^* edges.
4. So the component is an M -augmenting path. ■

This justifies the iterative scheme of Lecture 7: augment while an augmenting path exists; the matching we stop at is maximum.

Finding an augmenting path

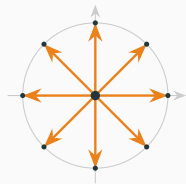
In the graph G_M of Lecture 7 we orient every unmatched edge from L to R and every matched edge from R to L . An augmenting path is then a directed path from a free left vertex to a free right vertex, so a depth-first search alternates on its own and G_M need not be built.

```
bool try_kuhn(int u, vector<vector<int>>& adj, vector<int>& mr, vector<bool>& used) {
    for (int v : adj[u]) { // step u -> v, an unmatched edge
        if (used[v]) continue;
        used[v] = true;
        if (mr[v] == -1 // v is free: the path ends here
            || try_kuhn(mr[v], adj, mr, used)) { // else step v -> mr[v], a matched edge
            mr[v] = u; return true; // flip this edge on the way back
        }
    }
    return false;
}
// for each u in L { used.assign(|R|, false); if (try_kuhn(u, ...)) ++ans; }
```

Correctness follows from Lemma 1 and Berge's Lemma: each successful search enlarges M by one, and we stop only when no augmenting path is left. The main loop runs one search per vertex of L and each search scans $O(|E|)$ edges, so the cost is $O(V \cdot E)$.

Inverting the Fourier matrix

Evaluating a polynomial at the n th roots of unity is multiplication by the Fourier matrix $M_n(\omega)_{jk} = \omega^{jk}$; interpolation is multiplication by its inverse. One fact about the roots of unity gives it.



the n th roots for $n = 8$, summing to 0

For $d \not\equiv 0 \pmod{n}$ the powers $\omega^{0d}, \dots, \omega^{(n-1)d}$ are unit vectors spaced evenly around the circle, repeated $\gcd(d, n)$ times each, so

$$\sum_{\ell=0}^{n-1} \omega^{\ell d} = \frac{(\omega^d)^n - 1}{\omega^d - 1} = 0,$$

the division being legal because $\omega^d \neq 1$. For $d = 0$ the sum is n .

Taking $d = j - k$, where $|j - k| < n$, gives

$$(M_n(\omega) M_n(\omega^{-1}))_{jk} = \sum_{\ell=0}^{n-1} \omega^{\ell(j-k)} = \begin{cases} n, & j = k \\ 0, & j \neq k \end{cases} \implies M_n(\omega)^{-1} = \frac{1}{n} M_n(\omega^{-1}).$$

ω^{-1} is again a primitive n th root of unity, so $M_n(\omega^{-1})$ is again a Fourier matrix: interpolation is the same algorithm as evaluation, run with ω^{-1} and divided by n .

What the inverse is doing

Row m of that identity says the m th coefficient comes back as an untwist and an average:

$$a_m = \frac{1}{n} \sum_{j=0}^{n-1} A(\omega^j) \omega^{-jm} = \frac{1}{n} \sum_{k=0}^{n-1} a_k \sum_{j=0}^{n-1} \omega^{j(k-m)}.$$

Evaluation twists each coefficient at its own rate: a_k contributes $a_k \omega^{jk}$, turning through $2\pi k/n$ as j advances by one. Untwisting by ω^{-jm} brings the term $k = m$ to a standstill and leaves every other term turning. Laid head to tail, the n unit steps of the inner sum look like this:



$k = m$: every step is 1
the chain runs straight, sum = n



$k \neq m$: the steps turn
the chain closes up, sum = 0

Standing still, the steps accumulate; turning, they close back on the start. Only $k = m$ survives, and the average returns a_m .

Multiplying two polynomials

```
void fft(vector<cd>& a, bool inv) {
    int n = a.size(); if (n == 1) return;
    vector<cd> ae(n/2), ao(n/2);
    for (int i = 0; i < n/2; i++)
        { ae[i] = a[2*i]; ao[i] = a[2*i+1]; }
    fft(ae, inv); fft(ao, inv);
    double ang = 2*PI/n * (inv ? -1 : 1);
    cd w(1), wn(cos(ang), sin(ang));
    for (int j = 0; j < n/2; j++) {
        a[j] = ae[j] + w*ao[j];
        a[j+n/2] = ae[j] - w*ao[j];
        if (inv) { a[j] /= 2; a[j+n/2] /= 2; }
        w *= wn;
    }
}
```

DPV Figure 2.9 with one extra argument: `inv` replaces ω by ω^{-1} , and halving at each of the $\log n$ levels performs the division by n .

To compute AB : evaluate, multiply pointwise, invert, and round.

Padding

$\deg(AB) \leq 2n - 2$, so the product has $2n - 1$ coefficients and n evaluation points cannot determine it. Pad both inputs with zeros to length m , the least power of two with $m \geq 2n - 1$.

Polynomial multiplication is convolution, which the transform turns into a pointwise product. The same routine underlies signal filtering, image blurring and template matching, and the convolution layers of vision networks.

A polynomial from its roots

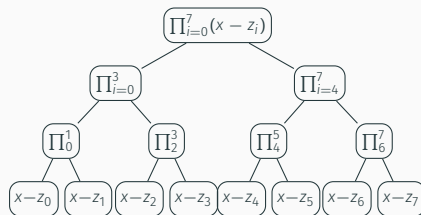
Given $z_0, \dots, z_{n-1}$, we want the coefficients of a polynomial vanishing exactly there. Since $P(z_j) = 0$ if and only if $(x - z_j)$ divides P , we may take $P(x) = \prod_{i=0}^{n-1} (x - z_i)$, of degree n . Only the order of multiplication is left to decide.

Multiplying the factors in one at a time costs $\Theta(i)$ at step i , hence $\Theta(n^2)$ overall: each step touches the whole accumulated polynomial but adds a single root, and FFT cannot help when one operand has two terms.

Pairing operands of equal size instead gives

$$T(n) = 2T(n/2) + O(n \log n) = O(n \log^2 n),$$

since each of the $\log n$ levels costs $2^j \cdot O(\frac{n}{2^j} \log \frac{n}{2^j}) = O(n \log n)$.



By induction the node spanning $[l, r)$ returns

$\prod_{i \in [l, r)} (x - z_i)$, so the root returns P . This construction will be useful in the assignment.

Reading

- Bipartite matchings: Section 1 of the Cornell CS6820 notes linked in Lecture 7.
- Fast Fourier transform: DPV Section 2.6.

Exercises

- Where does the proof of Berge's Lemma use bipartiteness? Does it still hold for general graphs?
- König's theorem: in a bipartite graph the maximum matching equals the minimum vertex cover. Prove it using the vertices reachable by alternating paths from the free vertices of L .
- Comparing two spanning trees the way we compared two matchings shows that an MST with distinct edge weights is unique. The same argument will be useful in the assignment.
- DPV 2.7, 2.8, 2.10.

Hopcroft–Karp improves $O(V \cdot E)$ to $O(E\sqrt{V})$ by augmenting along many shortest augmenting paths in each phase.